Árboles

Objetivos:

- Presentar la estructura no lineal más importante en computación
- Mostrar la especificación e implementación de varios tipos de árboles
- Algoritmos de manipulación de árboles

Contenido

1. Introducción
    1.1. Definición
    1.2. Terminología
    1.3. Recorridos
2. Árboles binarios
    2.1. Definición
    2.2. Especificación
    2.3. Implementación
3. Heap o Montículo binario
    3.1. Definición
    3.2. Especificación
    3.3. Implementación
4. Árboles binarios de búsqueda
    4.1. Definición
    4.2. Especificación
5. Árboles binarios equilibrados
    5.1. Árboles AVL
6. Árboles generales
    6.1. Especificación

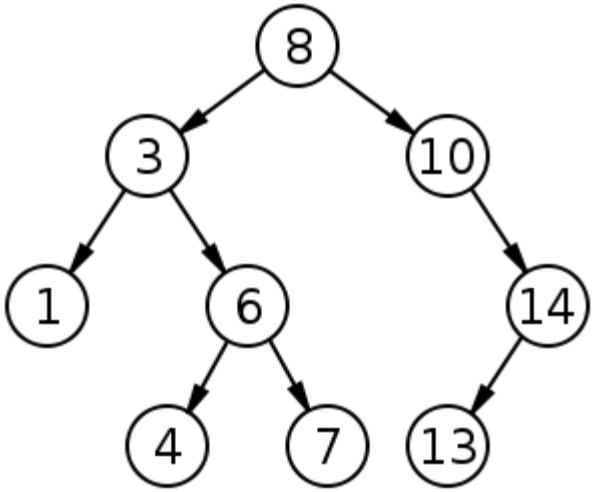
Definición

- También llamados BST (acrónimo del inglés *Binary Search Tree*)
- Estructura de datos básica para almacenar elementos que están clasificados siguiendo algún orden lineal.
- **Propiedades**:
  - Para todo nodo N del árbol,
    - Todos los valores de los nodos del subárbol izquierdo de N deben ser *menores* al valor del nodo N, y
    - Todos los valores de los nodos del subárbol derecho de N deben ser *mayores o iguales* al valor del nodo N
- Un recorrido en *inorden* del árbol proporciona una lista en orden ascendente de los valores almacenados en los nodos

Definición

- Ejemplo:

Un árbol binario de búsqueda de tamaño 9 y profundidad 3, con raíz 8 y hojas 1, 4, 7 y 13



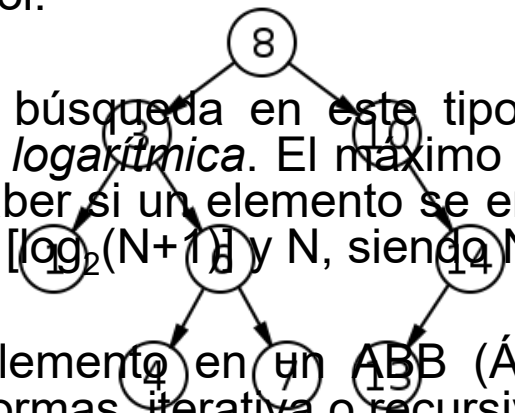
Búsqueda

Operaciones:

- ***Búsqueda:*** determinar si x está en el árbol
  - Si x = raíz ⇒ localizado
  - Si x < raíz ⇒ buscar en el subárbol izquierdo
  - Si x ≥ raíz ⇒ buscar en el subárbol derecho

- Consiste en acceder a la raíz del árbol;
  - si el elemento a localizar coincide con éste la búsqueda ha concluido con éxito;
  - si el elemento es menor se busca en el subárbol izquierdo y
  - si es mayor o igual, se busca en el subárbol derecho.

  Si se alcanza un nodo hoja y el elemento no ha sido encontrado se supone que no existe en el árbol.



- Cabe destacar que la búsqueda en este tipo de árboles es muy eficiente, representa una *función logarítmica*. El máximo número de comparaciones que necesitaríamos para saber si un elemento se encuentra en un árbol binario de búsqueda estaría entre $\lceil \log_2(N+1) \rceil$ y N, siendo N el número de nodos.

- La búsqueda de un elemento en un ABB (Árbol Binario de Búsqueda) se puede realizar de dos formas, iterativa o recursiva.

Inserción

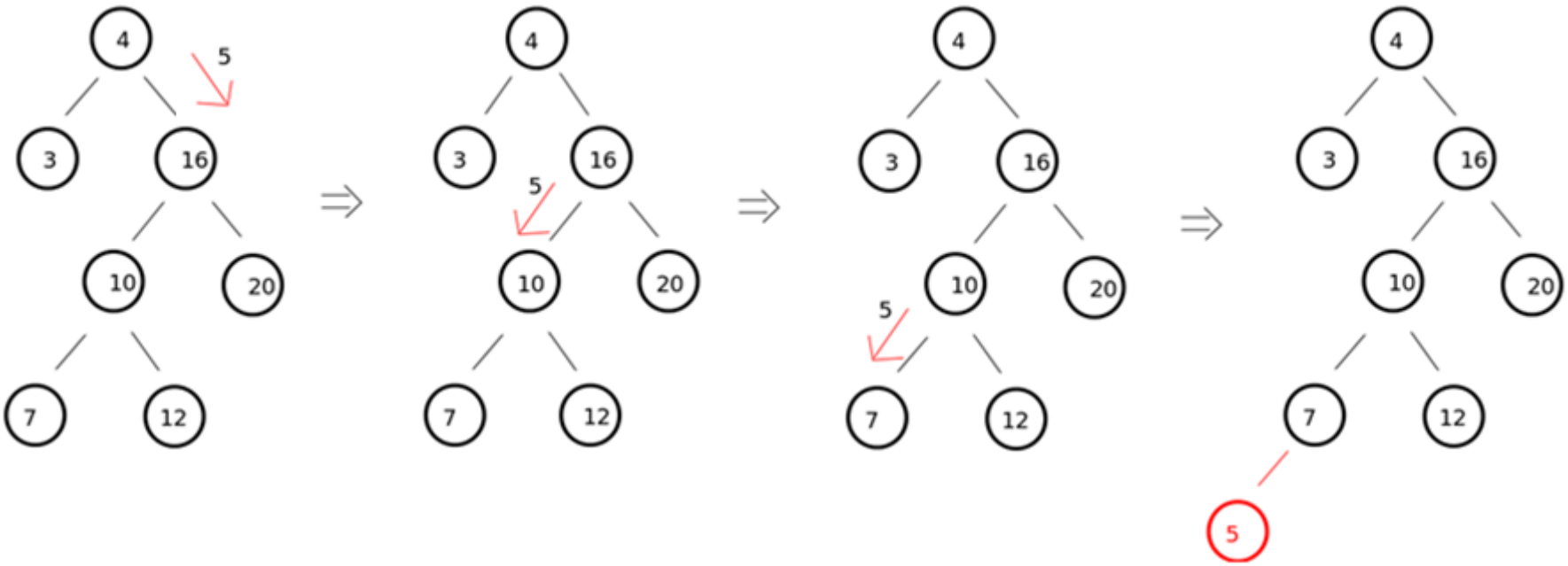
- ***Inserción***: añadir un nuevo elemento x al árbol
  - Avanzar en el árbol comparando x con raíz
  - Repetir el paso 1 hasta que el subárbol donde deba insertarse sea el árbol vacío
  - Insertar el elemento

La inserción es similar a la búsqueda y se puede dar una solución tanto iterativa como recursiva.

- Si tenemos inicialmente un árbol vacío se crea un nuevo nodo con el elemento a insertar.
- Si no lo está, se comprueba si el elemento dado es menor que la raíz del árbol inicial con lo que se inserta en el subárbol izquierdo y si es mayor o igual se inserta en el subárbol derecho.

Inserción

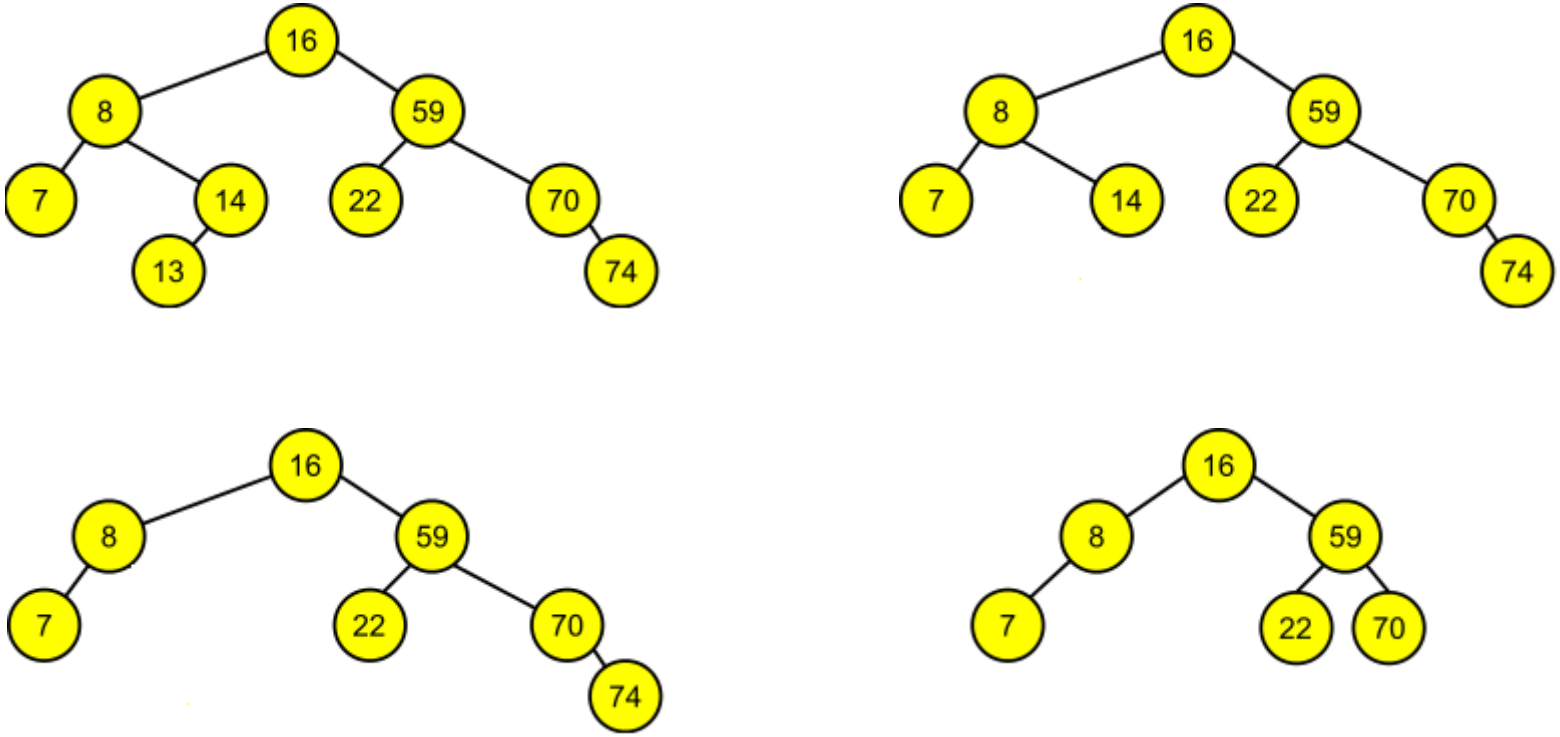
- Ejemplo de inserción: el entero 5



Eliminación

- ***Eliminación***: borrar un elemento x del árbol
  - Si x es hoja ⇒ se suprime x.

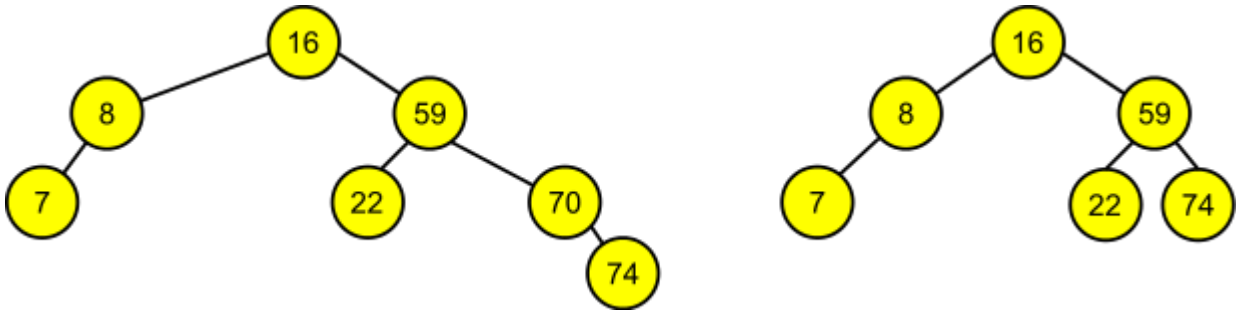
Ejemplo: borrado de los nodos 13, 14 y 74



Eliminación

- ***Eliminación***: borrar un elemento x del árbol
  - Si el elemento a borrar tiene un solo descendiente ⇒ se sustituye por ese descendiente.

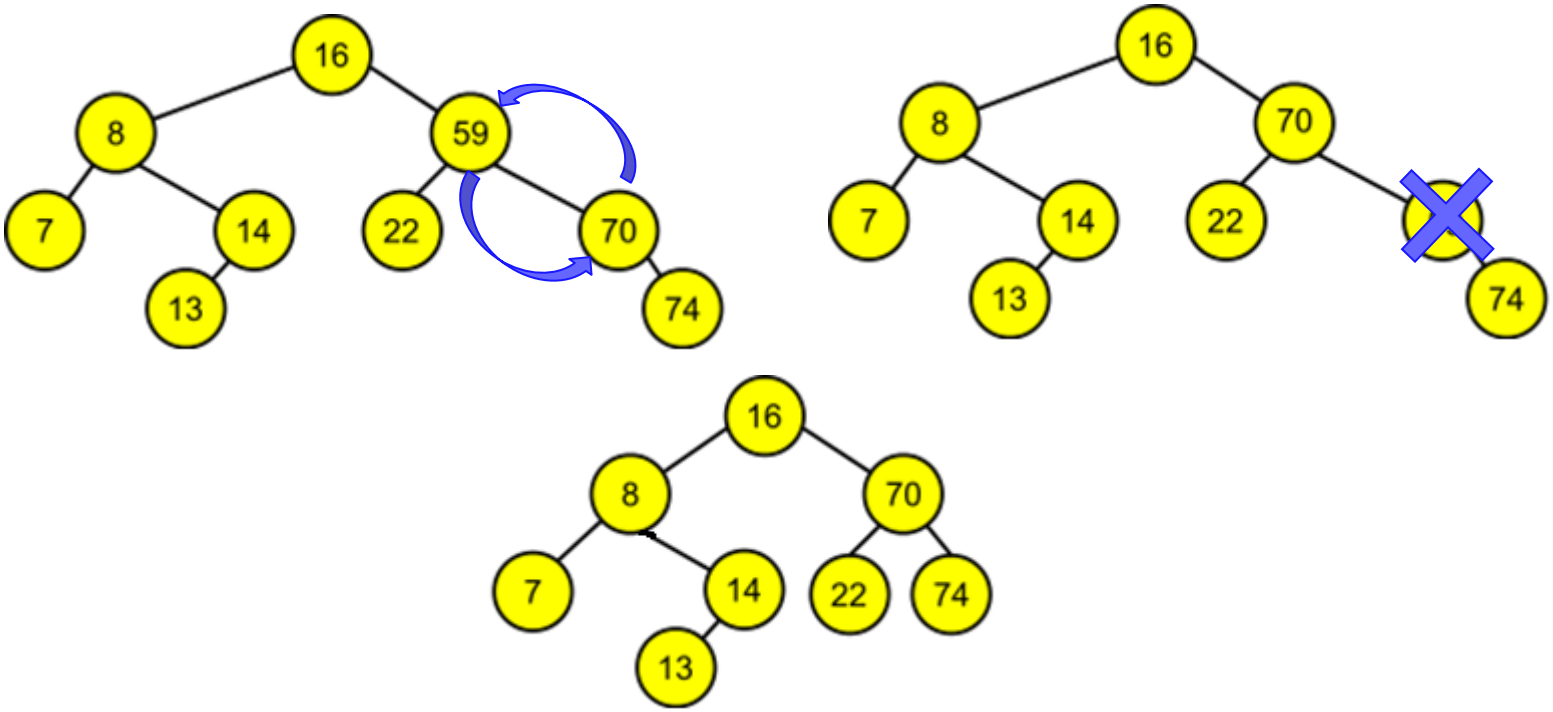
Ejemplo: borrado del nodo 70



Eliminación

- ***Eliminación***: borrar un elemento x del árbol
  - Si el elemento a borrar tiene dos descendientes ⇒ se sustituye por el menor elemento del subárbol derecho. Después se elimina el nodo correspondiente a dicho elemento

Ejemplo: borrado del nodo 59



TAD BinarySearchTree<T> - Especificación

```
BinarySearchTree<T> {
    // Declaración de tipos: BinarySearchTree
    // Características: Es un árbol binario donde para cada nodo se cumple la propiedad
    //                  de que todos los nodos con clave menor que la suya están en
    //                  su subárbol izquierdo y todos los nodos con clave mayor o igual
    //                  se encuentran en el subárbol derecho. No admite valores null.
    //                  Los objetos son modificables

    public BinarySearchTree();
        // Produce: un árbol binario de búsqueda vacío
    public T getRootValue() throws EmptyTreeException;
        // Produce: si el árbol está vacío lanza la excepción EmptyTreeException,
        //          sino devuelve el valor raíz del árbol
    public boolean contains(T value);
        // Produce: cierto si el valor está contenido en el árbol; falso en caso contrario.
    public void insert(T value) throws NullPointerException;
        // Modifica: this
        // Produce: si value es null lanza NullPointerException; en otro caso,
        //          añade value a this
    public void remove(T value)
        // Modifica: this
        // Produce: elimina una ocurrencia de value del árbol
```

TAD BinarySearchTree<T> - Especificación

```
public boolean hasLeftChild();
    //Produce: cierto si el árbol tiene hijo izquierdo; falso, en caso contrario.

public BinarySearchTree<T> getLeftChild() throws EmptyTreeException;
    // Produce: si el árbol está vacío lanza la excepción EmptyTreeException,
    //        sino devuelve el subárbol izquierdo.

public boolean hasRightChild ();
    //Produce: cierto si el árbol tiene hijo derecho; falso, en caso contrario.

public BinarySearchTree<T> getRightChild() throws EmptyTreeException;
    // Produce: si el árbol está vacío lanza la excepción EmptyTreeException,
    //        sino devuelve el subárbol derecho

public boolean isEmpty();
    // Produce: cierto si el árbol está vacío; falso en caso contrario.

public void clear();
    //Modifica: this
    //Produce: elimina todos los valores convirtiéndolo en un árbol vacío
}
```

TAD BinarySearchTree<T> - Implementación

- ¿Cómo comparar los elementos de los nodos?

  Solución: los elementos deben ser instancias de una clase que implemente la interface Comparable<E> existente en java.

```
public interface Comparable<E>{
        public int compareTo(E e);
}
```

- Paso 1: Definición interfaz

```
public interface BinarySearchTree <T extends Comparable<T>> {
    public T getRootValue() throws EmptyTreeException;
    public boolean contains(T value);
    public void insert(T value) throws NullPointerException;
    public void remove(T value);
    public boolean hasLeftChild();
    public BinarySearchTree<T> getLeftChild() throws EmptyTreeException;
    public boolean hasRightChild();
    public BinarySearchTree<T> getRightChild() throws EmptyTreeException;
    public boolean isEmpty();
    public void clear();
}
```

- Paso 2: Clase implemente la interfaz
  - Mediante estructuras enlazadas genéricas
    - **public class** LinkedBinarySearchTree<T **extends** Comparable<T>> **implements** BinarySearchTree<T>

Ejercicios

Ejercicios autoevaluación:

- Para practicar el manejo de árboles binarios de búsqueda, pincha en el siguiente enlace: https://visualgo.net/en/bst
- Dibuja paso a paso el árbol binario de búsqueda que resulta a partir de la siguiente entrada de datos: 46, 73, 25, 36, 62, 12, 90, 5, 70.
- Una vez dibujado elimina los siguientes elementos: 46, 90, 73, 25.
- Genera el árbol binario de búsqueda para la siguiente secuencia de números: 8, 9, 11, 15, 19, 20, 21, 7, 6, 3, 2, 1. Analiza y describe lo que sucede.